

# Introduction to MySQL Joins

## What is a JOIN?

In relational databases like MySQL, data is often split into multiple tables to keep it organized and avoid duplication. A JOIN is used to combine rows from two or more tables based on a related column between them.

### The Problem: Why do we need Joins?

Imagine we are building a School Management System. We have two tables: one for students and one for classes.

**Table 1: students**

| student_id | name  | class_id |
|------------|-------|----------|
| 1          | Kamal | 101      |
| 2          | Nimal | 102      |
| 3          | Amal  | 101      |

**Table 2: classes**

| class_id | class_name   |
|----------|--------------|
| 101      | Grade 10 - A |
| 102      | Grade 10 - B |

**The Goal:** We want to see a list of student names alongside their actual class names (e.g., "Kamal - Grade 10 - A"). If we run `SELECT * FROM students;`, we only get the `class_id` (101, 102), not the actual name of the class. To get the class name, we must connect (or "join") the `students` table with the `classes` table using the column they both share: `class_id`.

## 1. INNER JOIN

An `INNER JOIN` selects records that have matching values in **both** tables. If a row does not have a match in the other table, it is left out of the results.

**The Query:**

```
SELECT students.name, classes.class_name
```

```
FROM students
INNER JOIN classes ON students.class_id = classes.class_id;
```

(Note: The *ON* keyword tells MySQL exactly which columns to use to link the two tables together.)

### The Result:

```
+-----+-----+
| name  | class_name |
+-----+-----+
| Kamal | Grade 10 - A |
| Nimal | Grade 10 - B |
| Amal  | Grade 10 - A |
+-----+-----+
```

Notice how only the students who have a valid *class\_id* that matches the *classes* table are displayed.

---

## 2. LEFT JOIN

What happens if we have data that doesn't perfectly match? Imagine a new student named **Sunil** joins the school today. He is added to the *students* table, but a class hasn't been assigned to him yet, so his *class\_id* is *NULL*.

### Updated students Table:

```
+-----+-----+-----+
| student_id | name  | class_id |
+-----+-----+-----+
| 1          | Kamal | 101      |
| 2          | Nimal | 102      |
| 3          | Amal  | 101      |
| 4          | Sunil | NULL     |
+-----+-----+-----+
```

If we use an *INNER JOIN* now, Sunil will not appear in the results because his *class\_id* does not match anything in the *classes* table.

To see **all** students, whether they have a class or not, we use a *LEFT JOIN*. A *LEFT JOIN* returns all records from the left table (*students*), and the matched records from the right table (*classes*). If there is no match, the result is *NULL* on the right side.

### The Query:

```
SELECT students.name, classes.class_name
FROM students
LEFT JOIN classes ON students.class_id = classes.class_id;
```

### The Result:

```
+-----+-----+
| name  | class_name |
+-----+-----+
```

```
+-----+-----+
| Kamal | Grade 10 - A |
| Nimal | Grade 10 - B |
| Amal  | Grade 10 - A |
| Sunil | NULL          |
+-----+-----+
```

*Notice that Sunil is now included in our results, and his class name shows as NULL because he hasn't been assigned one yet.*

[illegible][illegible]

# Types of SQL Joins

In this lesson, we will explore the different types of SQL Joins using our **School Management System** example. We will use two tables for these examples: students (Left Table) and classes (Right Table).

## 1. INNER JOIN

An **INNER JOIN** returns only the rows that have matching values in **both** tables.

- **How to explain to students:** "Imagine we only want to see students who have actually been assigned a class, along with the class name. Students without classes, or classes without students, are completely ignored."
- **The SQL Query:**

```
SELECT students.name, classes.class_name
FROM students
INNER JOIN classes ON students.class_id = classes.class_id;
```

## 2. LEFT JOIN (LEFT OUTER JOIN)

A **LEFT JOIN** returns **all** records from the left table (`students`), and the matched records from the right table (`classes`). If there is no match, the result is `NULL` on the right side.

- **How to explain to students:** "We want to see the details of **every student** in the school. If a new student (like Sunil) hasn't been assigned a class yet, his class name will show as NULL, but his name will still definitely appear on our list."
- **The SQL Query:**

```
SELECT students.name, classes.class_name
FROM students
```

```
LEFT JOIN classes ON students.class_id = classes.class_id;
```

---

### 3. RIGHT JOIN (RIGHT OUTER JOIN)

A **RIGHT JOIN** is the exact opposite of a Left Join. It returns **all** records from the right table (classes), and the matched records from the left table (students). If there is no match, the result is NULL on the left side.

- **How to explain to students:** "Now we want a list of **every class** in the school. Even if a newly created class has zero students enrolled in it yet (e.g., Grade 10 - C), the class will still appear on the list, and the student name next to it will show as NULL."
- **The SQL Query:**

```
SELECT students.name, classes.class_name
FROM students
RIGHT JOIN classes ON students.class_id = classes.class_id;
```

---

### 4. FULL OUTER JOIN

A **FULL OUTER JOIN** returns **all records** when there is a match in either the left or right table. It combines the results of both Left and Right joins.

- **How to explain to students:** "If we want to see absolutely everything in one place students with classes, students without classes, and classes without students - we use a Full Outer Join."
- *Note for MySQL:* MySQL does not have a direct FULL OUTER JOIN keyword. Instead, we use the UNION keyword to combine a LEFT JOIN and a RIGHT JOIN together.
- **The SQL Query (for MySQL):**

```
SELECT students.name, classes.class_name
FROM students
LEFT JOIN classes ON students.class_id = classes.class_id
UNION
SELECT students.name, classes.class_name
FROM students
RIGHT JOIN classes ON students.class_id = classes.class_id;
```

---

### 5. CROSS JOIN

A **CROSS JOIN** returns the Cartesian product of the two tables. This means it returns all possible combinations of rows from both tables. It does not use an ON condition.

- **How to explain to students:** "Think of this as multiplication. If we have 4 students in the first table and 2 classes in the second table, the result will be a list of 8 rows (4 x 2). It pairs every single student with every single class available."

- **The SQL Query:**

```
SELECT students.name, classes.class_name
FROM students
CROSS JOIN classes;
```

## 6. SELF JOIN

A **SELF JOIN** is a regular join, but the table is joined with itself. (There is no special **SELF JOIN** keyword; we just use standard joins).

- **How to explain to students:** "Imagine we want to pair up students who are in the exact same class. To find friends like Kamal and Amal who are both in 'Grade 10 - A' (class\_id 101), we can join the students table to itself."
- **The SQL Query:**

SQL

```
SELECT A.name AS Student1, B.name AS Student2, A.class_id
FROM students A, students B
WHERE A.student_id <> B.student_id
AND A.class_id = B.class_id;
```

(Note: We use letters like 'A' and 'B' as "Aliases" so the database knows which version of the *students* table we are referring to).

### MySQL JOINS Explained with Venn Diagrams

Assume two tables: **A** (left table) and **B** (right table)

#### 1. INNER JOIN

Returns only the matching rows that exist in both tables.

```
SELECT * FROM A
INNER JOIN B ON A.id = B.a_id;
```

#### 2. LEFT JOIN (or LEFT OUTER JOIN)

Returns all rows from the left table (A) and the matching rows from the right table (B). If no match, NULLs for columns from B.

```
SELECT * FROM A
LEFT JOIN B ON A.id = B.a_id;
```

#### 3. RIGHT JOIN (or RIGHT OUTER JOIN)

Returns all rows from the right table (B) and the matching rows from the left table (A). If no match, NULLs for columns from A.

```
SELECT * FROM A
RIGHT JOIN B ON A.id = B.a_id;
```

#### 4. LEFT JOIN EXCLUDING INNER JOIN (LEFT ANTI JOIN)

Returns rows that exist in the left table (A) but have no match in the right table (B).

```
SELECT * FROM A
LEFT JOIN B ON A.id = B.a_id
WHERE B.a_id IS NULL;
```

#### 5. RIGHT JOIN EXCLUDING INNER JOIN (RIGHT ANTI JOIN)

Returns rows that exist in the right table (B) but have no match in the left table (A).

```
SELECT * FROM A
RIGHT JOIN B ON A.id = B.a_id
WHERE A.id IS NULL;
```

#### 6. FULL OUTER JOIN

Returns all rows when there is a match in either table. If no match, NULLs for the missing side. (MySQL doesn't support FULL OUTER JOIN directly.)

```
SELECT * FROM A
LEFT JOIN B ON A.id = B.a_id
UNION
SELECT * FROM A
RIGHT JOIN B ON A.id = B.a_id;
```

**Legend**

- Rows from table A
- Rows from table B
- Matching rows in both A and B

💡 JOIN ... ON specifies the condition for matching rows between the two tables.

